Java Silver（SE 17・1Z0-825-JPN）　6 例外処理　#13

例外処理

どの例外に処理が必要か。どの順に動き、何が最後に残るか

各ページの下に、根拠にした Oracle の資料（JLS 11・14.18・14.20、Java SE 17 API Throwable・Exception・RuntimeException・Error・AutoCloseable、Java Tutorials）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 6（例外処理）

1 例外の種類
チェック例外は処理が必須。RuntimeException 系と Error 系は不要

2 throw・throws・カスタム例外
Exception を継承すればチェック、RuntimeException なら非チェック

3 try-catch-finally
catch は上から 1 つだけ。finally は必ず。マルチキャッチは |

4 try-with-resources
AutoCloseable を自動で close。逆順に閉じ、catch・finally はそのあと

チェックリストの項目	このPDFのページ
チェック例外・非チェック例外・エラーの違い	第1章
カスタム例外の作成	第1章
try-catch 文による例外処理	第2章
multi-catch 句の使用	第2章
try-with-resources 文による例外処理	第3章

> **試験の多くは「コンパイルできるか」と「何が表示されるか」。**前者はチェック例外と catch の並び、後者は finally と close の順番で決まる。

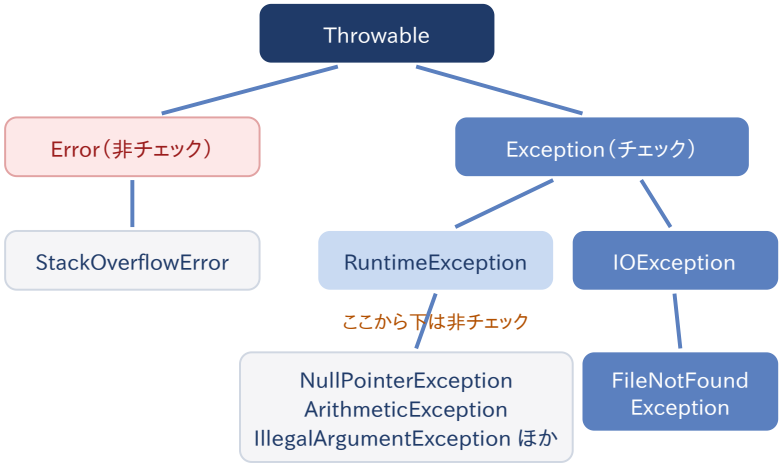
出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JLS 11 Exceptions

第1章

例外の種類とカスタム例外

どのクラスを継承しているかで、処理が必須かどうかが決まる。

例外クラスの階層



よく出る非チェック例外	起きる場面（JDK 17 で確認）
ArithmeticException	整数を 0 で割る（/ by zero）
ArrayIndexOutOfBoundsException	範囲外の添字（Index 1 out of bounds for length 1）
NullPointerException	null のメソッドを呼ぶ
ClassCastException	中身と合わない型へキャスト
NumberFormatException	Integer.parseInt("1a")
StackOverflowError	終わらない再帰（Error）

> **Error は Exception の仲間ではない。**catch (Exception e) では StackOverflowError を受け取れない。catch (Error e) か catch (Throwable t) が必要。

出典：JLS 11.1.1（the checked exception classes are Throwable and all its subclasses other than RuntimeException and its subclasses and Error and its subclasses）／Java SE 17 API Error／結果は JDK 17 で確かめたもの

チェック例外は「受け取るか、宣言するか」

```
static void read() throws IOException { … }
// どちらかが必要
try { read(); } catch (IOException e) { … }   // ① 受け取る
void main2() throws IOException { read(); } // ② 宣言して呼び出し元へ

read();   // どちらもない
```

```
error: unreported exception IOException;
  must be caught or declared to be thrown
```

… は省略（Claude が作成）。main に throws を書くと、例外は JVM まで伝わってプログラムが止まる。

コード	JDK 17
throws なしで `throw new Exception("x")`	unreported exception Exception; …
throws なしで `throw new IllegalStateException()`	OK（非チェック）
try で何も投げないのに `catch (IOException e)`	exception IOException is never thrown in body of corresponding try statement
try で何も投げないのに `catch (Exception e)`	OK
`throw …;` の直後に文	unreachable statement

catch (Exception e) は特別扱い。RuntimeException も含むので、try で何も投げなくても書ける。

出典：JLS 11.2.3（It is a compile-time error if a method or constructor body can throw some exception class E when E is a checked exception class and E is not a subclass of some class declared in the throws clause of the method or constructor）／結果は JDK 17 で確かめたもの

カスタム例外とメッセージ

```
class InsufficientFundsException extends Exception {
  InsufficientFundsException(String msg) { super(msg); }
}
static int balance = 100;
static void withdraw(int amount) throws InsufficientFundsException {
  if (amount > balance)
    throw new InsufficientFundsException("short by " + (amount - balance));
  balance -= amount;
}
// withdraw(30); withdraw(100); → catch で表示
```

```
short by 30 / 70 / InsufficientFundsException: short by 30
```

表示は e.getMessage() + " / " + balance + " / " + e（Claude が作成）。

継承するクラス	投げるメソッドの throws
extends Exception	必要（ないと unreported exception AppException）
extends RuntimeException	不要

式	結果（JDK 17）
new Exception("x").getMessage()	x
new Exception("x").toString()	java.lang.Exception: x
new Exception().getMessage()	null
new Exception().toString()	java.lang.Exception

出典：Java SE 17 API Throwable.toString()（The result is the concatenation of: the name of the class of this object ": " (a colon and a space) the result of invoking this object's getLocalizedMessage() method／If getLocalizedMessage returns null, then just the class name is returned）／Exception／結果は JDK 17 で確かめたもの

第2章

try-catch-finally とマルチキャッチ

catch は上から 1 つだけ。finally は何があっても最後に。

try-catch-finally の流れ

```
try {
  System.out.print("A ");
  int x = 10 / 0;           // ArithmeticException
  System.out.print("B ");        // 飛ばされる
} catch (ArithmeticException e) {
  System.out.print("C ");
} finally {
  System.out.print("D ");
}
System.out.println("E");
```

```
A C D E
```

10 / 0 の行を消すと A B D E（catch は動かず、finally は動く）。



try のあとには catch か finally が必要。try だけだと 'try' without 'catch', 'finally' or resource declarations。

出典：JLS 14.20（TryStatement: try Block Catches／try Block [Catches] Finally／TryWithResourcesStatement）・14.20.2（If execution of the try block completes normally, then the finally block is executed）／結果は JDK 17 で確かめたもの

catch の順番とマルチキャッチ

書き方	JDK 17
`catch (Exception e)` → `catch (IOException e)`	exception IOException has already been caught
`catch (IOException e)` → `catch (Exception e)`	OK（サブクラスが先）
`catch (FileNotFoundException | IOException e)`	Alternatives in a multi-catch statement cannot be related by subclassing
マルチキャッチの `e = …`	multi-catch parameter e may not be assigned
`catch (RuntimeException e)` で Exception を受ける	受け取れない（unreported exception Exception）

```
String[] a = { "1", "x" };
for (String s : a) {
  try {
    int n = Integer.parseInt(s);
    System.out.print(10 / (n - 1) + " ");
  } catch (NumberFormatException | ArithmeticException e) {
    System.out.print(e.getClass().getSimpleName() + " ");
  }
}
```

```
ArithmeticException NumberFormatException
```

マルチキャッチの e は暗黙に final。ふつうの catch の e には代入できる。
並べた型は継承関係のないものだけ。

出典：JLS 11.2.3（a preceding catch clause of the immediately enclosing try statement can catch E1 or a superclass of E1）／JLS 14.20（An exception parameter of a multi-catch clause is implicitly declared final if it is not explicitly declared final）／結果は JDK 17 で確かめたもの

finally と return

```
static int f1() {
  try { return 1; } finally { return 2; }
}                                    // → 2
static int f2() {
  int x = 1;
  try { return x; } finally { x = 5; }
}                                    // → 1
static StringBuilder f3() {
  StringBuilder sb = new StringBuilder("a");
  try { return sb; } finally { sb.append("b"); }
}                                    // → ab
```

場合	結果のしくみ
finally で return	finally の値が返る。try の return や例外は捨てられる
finally で基本型の変数を変える	戻り値は return の時点で決まっているので変わらない
finally で参照先を変える	返るのは同じオブジェクトなので、変更が見える
System.exit(0)	JVM が止まるので finally は動かない

> **呼び出しの順にも注意。**`println(f("abc") + " " + f(null))` は、f が 2 回とも動いて（finally の表示も 2 回）から println が 1 回表示する。

出典：JLS 14.20.2（If the finally block completes abruptly for reason S, then the try statement completes abruptly for reason S）／JLS 11.3（the reason for the abrupt completion of the try block is discarded and the new reason for abrupt completion is propagated from there）／結果は JDK 17 で確かめたもの

例外の伝播と finally

catch の中で別の例外（Claude が作成）

```
try {
  try {
    throw new IllegalStateException("1");
  } catch (IllegalStateException e) {
    System.out.print("catch1 ");
    throw new IllegalArgumentException("2");
  } finally {
    System.out.print("finally1 ");
  }
} catch (IllegalArgumentException e) {
  System.out.println("outer " + e.getMessage());
}
```

```
catch1 finally1 outer 2
```

どこでも受け取られない

```
try {
  System.out.println("try");
  throw new IllegalArgumentException("boom");
} finally {
  System.out.println("finally");
}
```

```
try
finally
Exception in thread "main" java.lang.IllegalArgumentException: boom
```

finally は、例外が外へ出ていく前に動く。そのあと呼び出し元へさかのぼり、だれも受け取らなければ main が終わる。

出典：JLS 14.20.2（If the catch block completes abruptly for reason R, then the finally block is executed）・14.18（If no such try statement is found, then execution of the thread … that executed the throw is terminated）／結果は JDK 17 で確かめたもの

第3章

try-with-resources

開いたものを、開いた逆の順に、自動で閉じる。

リソースが開いて閉じる順番

```
class R implements AutoCloseable {
  String n;
  R(String n) { this.n = n; print("open" + n + " "); }
  public void close() { print("close" + n + " "); }
}
try (R a = new R("A"); R b = new R("B")) {
  print("body ");
  throw new RuntimeException("x");
} catch (RuntimeException e) {
  print("catch ");
} finally {
  print("finally");
}
```

```
openA openB body closeB closeA catch finally
```

print は System.out.print の略として書いた。



catch・finally の時点で、リソースはもう閉じている

場合	close() は
本体が正常に終わる	**逆順に呼ばれる**
本体で例外	**逆順に呼ばれてから catch へ**
`try (R r = null)`	**呼ばれない（例外にもならない）**

出典：JLS 14.20.3（Resources are closed in the reverse order from that in which they were initialized／A resource is closed only if it initialized to a non-null value）・14.20.3.2（all resources will have been closed (or attempted to be closed) by the time the finally block is executed）／結果は JDK 17 で確かめたもの

close() の例外と、抑制された例外

```
class R implements AutoCloseable {
  public void close() throws Exception {
    throw new Exception("close");
  }
}
try (R r = new R()) {
  throw new IllegalStateException("body");
} catch (Exception e) {
  System.out.println(e.getMessage() + " "
    + e.getSuppressed().length + " "
    + e.getSuppressed()[0].getMessage());
}
```

```
body 1 close
```

本体の例外 body（主役）　close の例外（抑制）

catch に届くのは body。close の例外は
body.getSuppressed() に入る

close() の宣言	呼び出し側（JDK 17）
close() throws Exception で catch なし	unreported exception Exception; …（exception thrown from implicit call to close() on resource variable 'r'）
close()（throws なし）	catch も throws も不要

AutoCloseable の close() は throws Exception。実装するときに throws を外したり狭めたりできる（#11 のオーバーライドの決まり）。

出典：JLS 14.20.3（Such an exception is suppressed if an exception was thrown previously by an initializer, the try block, or the closing of a resource）／Throwable.getSuppressed()／AutoCloseable／結果は JDK 17 で確かめたもの

リソースに書けるもの

書き方	JDK 17
AutoCloseable を実装しないクラス（close() はある）	incompatible types: try-with-resources not applicable to variable type
本体で `r = new R();`	auto-closeable resource r may not be assigned
`R r = new R(); try (r) { … }`	OK（Java 9 から）
r に 2 回代入してから `try (r)`	variable r used as a try-with-resources resource neither final nor effectively final
catch も finally もない try-with-resources	OK

	try-catch-finally	try-with-resources
catch・finally	どちらか必須	なくてよい
後片付け	finally に自分で書く	close() が自動
片付けの例外	本体の例外を上書きしうる	抑制された例外として残る

リソース変数は final 扱い。( ) で宣言したものは暗黙に final、既存の変数は final か実質的に final でなければならない。

出典：JLS 14.20.3（must be a subtype of AutoCloseable, or a compile-time error occurs／A local variable declared in a resource specification is implicitly declared final if it is not explicitly declared final／An existing variable referred to in a resource specification must be a final or effectively final variable）／Java Tutorials／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
コンパイルできるか（throw）	チェック例外なら catch か throws。RuntimeException 系・Error 系は不要
コンパイルできるか（catch）	サブクラスが先。投げられないチェック例外は catch 不可。マルチキャッチに親子を並べない
表示の順番	例外のあとの try の文は飛ばす → catch 1 つ → finally → try 文のあと
戻り値	finally の return が勝つ。基本型の変更は戻り値に効かない
try-with-resources の順番	開く順 → 本体 → 逆順に close → catch → finally
例外のメッセージ	getMessage はメッセージだけ、toString は「パッケージ付きクラス名: メッセージ」
カスタム例外	Exception を継承→throws 必要。RuntimeException を継承→不要

出典：このPDFの各ページの資料

まとめ

例外の種類
Exception（RuntimeException 以外）はチェック例外

throw・throws
チェック例外は受け取るか宣言する。カスタム例外は継承元で決まる

try-catch-finally
catch はサブクラスから。finally は必ず。マルチキャッチの e は final

try-with-resources
AutoCloseable。逆順に close。close の例外は抑制される

こんなとき	結果
throws なしで throw new Exception()	コンパイルエラー
catch (Exception) のあとに catch (IOException)	コンパイルエラー
try で return 1、finally で return 2	2
リソース A, B の close 順	B → A
catch (Exception) で StackOverflowError	受け取れない

Java Silver の 13 分野はこれで全部。各分野の問題演習と、まちがえた問題の解き直しで仕上げる。

出典：このPDFの各ページの資料